

ChargerDoc Theme 2 Triage — Codebase Summary

What This Project Does

ChargerDoc is an AI-assisted EV (Electric Vehicle) charger troubleshooting platform. Its core job is to accept photographic evidence of a malfunctioning EV charging installation, use a Gemini Vision Language Model (VLM) to perceive what's in the image, map the findings to a deterministic rule set ("Theme 2"), and route the result to either:

- the **customer** (with self-service instructions), or
- **after-sales team** `AS_TEAM_01` (for hardware faults needing a technician).

The system handles three physical components:

Component	Examples of Observations
Charger	Red light, blinking red light (with flash-count error log), no light, serial/brand visible
EVDB (Electric Vehicle Distribution Board)	Single-phase / three-phase, MCB/RCCB tripped, missing or wrong-spec breakers
Isolator	Switch ON or switch OFF (open circuit)

The project is a **Round 2 (Theme 2)** prototype; Round 1 (pgvector, KB retrieval, `issue_family`, `resolver_tier`) is fully archived and not part of the live runtime.

System Architecture

```
Browser
→ Vercel (Next.js 15 frontend)
→ Next.js API proxy routes
→ FastAPI backend (Cloud Run / local Uvicorn)
→ Gemini VLM ← image/app-screenshot perception
→ Deterministic Theme 2 rule mapper
→ Neon / local PostgreSQL ← incidents + triage_audits
→ Google Cloud Storage / local disk ← uploaded evidence
```

Live triage path:

```
Upload image
→ Theme 2 perception (VLM → heuristic fallback)
→ Optional EV app screenshot parsing
→ Organizer rule mapping
→ Proof/follow-up prompts
→ Customer guidance or after-sales routing
→ Audit saved to DB / history view
```

Directory-by-Directory Breakdown

/backend/ — FastAPI Python Application

The backend is a Python 3.11+ package named `omnitriage-backend`, served by Uvicorn.

/backend/app/main.py

The FastAPI application entry point.

- Registers HTTP middleware for structured JSON request logging with request IDs and timing.
- Mounts `/uploads` as a static file endpoint for local evidence images.
- Runs `init_db()` on startup to create Postgres tables.
- Exposes these REST API endpoints:

Endpoint	Method	Purpose
<code>/api/v1/health</code>	GET	Returns runtime status, VLM provider, rule version
<code>/api/v1/sites</code>	GET	Lists configured EV charging sites
<code>/api/v1/demo/scenarios</code>	GET	Returns pre-built demo scenarios for the UI
<code>/api/v1/incidents</code>	GET	Lists all incident history
<code>/api/v1/incidents/{id}</code>	GET	Gets a single incident with audit trail
<code>/api/v1/uploads</code>	POST	Stores a base64-encoded photo, returns storage key
<code>/api/v1/evidence/{key}</code>	GET	Retrieves stored evidence bytes
<code>/api/v1/intake/preview</code>	POST	Assesses image quality, generates follow-up questions
<code>/api/v1/triage</code>	POST	Main endpoint — runs full Theme 2 triage pipeline

/backend/app/core/ — Data Models & Static Data

File	Purpose
<code>models.py</code>	All Pydantic v2 data models: <code>IncidentInput</code> , <code>PhotoEvidence</code> , <code>Theme2VisualExtraction</code> , <code>Theme2PerceptionAssessment</code> , <code>CompetitionOutput</code> , <code>Theme2TriageResult</code> , <code>SiteCapabilityProfile</code> , <code>DemoScenario</code> . Also defines key literal types: <code>InputComponent</code> , <code>ObservationResultV2</code> , <code>FaultTypeV2</code> , <code>RecipientType</code> .

File	Purpose
<code>data.py</code>	Loaders for <code>data/sites/sites.json</code> and <code>data/demo/scenarios.json</code> into Pydantic models.

Key model flow:

```
IncidentInput
→ Theme2PerceptionAssessment (VLM / heuristic output)
→ Theme2VisualExtraction (structured field extraction)
→ CompetitionOutput (organizer-facing result)
→ Theme2TriageResult (full response)
```

`/backend/app/services/` — *Business Logic Services*

File	Purpose
<code>gemini_client.py</code>	Thin wrapper around <code>google-genai</code> SDK to call Gemini VLM with a text+image prompt. Handles API key loading and basic error reporting.
<code>theme2_perception.py</code>	Largest file (75 KB). The core VLM perception engine. Builds a detailed prompt asking Gemini to identify the component (charger/EVDB/isolator), indicator state, MCB/RCCB specs, bounding boxes, visible text, etc. Falls back to heuristic classification if Gemini is unavailable or returns a malformed response.
<code>theme2_mapper.py</code>	Deterministic rule mapper. Takes a <code>Theme2PerceptionAssessment</code> and maps it to a <code>CompetitionOutput</code> using <code>theme2_rules.json</code> . Includes: safety escalation overrides (smoke, sparks, burn), repeated MCB-trip escalation, no-light unresolved escalation, EVDB spec review (amp/pole/RCCB-type mismatch detection), and blinking-red error-log key extraction (flash count 6–9 → specific fault).
<code>theme2_triage.py</code>	Orchestrator: calls perception → mapper → builds follow-up prompt list → assembles debug info → returns <code>Theme2TriageResult</code> .
<code>theme2_rules.py</code>	Thin loader for <code>data/round2/theme2_rules.json</code> ; exposes helpers <code>get_theme2_rule()</code> , <code>get_error_log_rule()</code> , <code>rule_fault_type()</code> , <code>rule_recipient()</code> .

File	Purpose
<code>intake.py</code>	Image quality assessment (file size, hint text heuristics) and follow-up question generation for the <code>/intake/preview</code> endpoint.
<code>storage.py</code>	Dual-mode storage: local filesystem or Google Cloud Storage (GCS). Handles saving/reading base64-encoded evidence with a stable storage key.
<code>history.py</code>	Thin wrapper over <code>db/persistence.py</code> for listing and retrieving incident history. Filters out legacy Round 1 incidents.

`/backend/app/db/` — *Database Persistence*

File	Purpose
<code>persistence.py</code>	PostgreSQL persistence via <code>psycopg</code> v3. Creates two tables on startup: <code>incidents</code> (one row per triage session) and <code>triage_audits</code> (one row per pipeline stage — <code>intake_preview</code> , <code>theme2_perception</code> , <code>theme2_mapping</code> , <code>triage_result</code>). Provides <code>save_incident()</code> , <code>update_incident()</code> , <code>save_audit()</code> , <code>list_recent_incidents()</code> , <code>get_incident_by_id()</code> . Joins audit rows to reconstruct the last triage result on history reads.

`/backend/tests/`

pytest test suite for the backend. CI runs `py -m pytest -q` and `py -m pyright`.

`/frontend/` — Next.js 15 TypeScript Application

The frontend is `omnitriage-frontend`, a Next.js 15 / React 19 app deployed on Vercel.

`/frontend/app/` — *Next.js App Router Pages*

Route	Purpose
<code>/ (page.tsx)</code>	Landing page (renders)
<code>/upload</code>	Multi-step image upload flow: select site/charger, drag-drop photo, image compression, call <code>/api/v1/uploads</code> then <code>/api/v1/intake/preview</code>

Route	Purpose
/result	Triage result display: calls <code>/api/v1/triage</code> , shows <code>CompetitionOutput</code> with bounding boxes, confidence pill, follow-up prompts, EV app screenshot upload option for red-light cases
/guidance	Customer self-service guidance page (for <code>recipient_type: customer</code> results)
/escalation	After-sales escalation page (for <code>recipient_type: after_sales_team</code> results)
/questions	Follow-up questions UI for additional evidence collection
/confirmation	Post-triage confirmation screen
/safety	Safety-critical escalation flow
/quality	Image quality check page
/intake	Intake preview page
/history	Incident history list view
/demo	Demo mode with pre-loaded scenarios from <code>/api/v1/demo/scenarios</code>
/api/	Next.js API proxy routes — forward calls to the FastAPI backend, adding server-side <code>API_BASE_URL</code>

/frontend/components/ — Reusable React Components

Subdirectory	Contents
triage/	17 focused result-display components: <code>ResultVerdictCard</code> , <code>EvidencePanel</code> (bounding box overlay), <code>ConfidencePill</code> , <code>FollowUpControl</code> , <code>ProofRequiredCard</code> , <code>DecisionChain</code> , <code>RouteBadge</code> , <code>StatusTimeline</code> , <code>UploadDropzone</code> , etc.
landing/	Landing page hero and feature section components
layout/	Site header/footer, navigation wrappers
ui/	Shared primitive UI components (built on Radix UI + shadcn/ui conventions)
figma/	Figma-exported or design-token components
site-shell.tsx	Top-level shell wrapper providing theme and layout context

/frontend/lib/ — TypeScript Utilities

File	Purpose
api.ts	All typed API client functions: <code>uploadPhoto()</code> , <code>intakePreview()</code> , <code>runTriage()</code> , <code>getIncidents()</code> , etc. Reads <code>NEXT_PUBLIC_API_BASE_URL</code> .
theme2-result-fields.ts	Maps <code>ObservationResultV2</code> / <code>FaultTypeV2</code> / <code>RecipientType</code> enum values to human-readable labels, icons, and color tokens for the result UI.
theme2-result-state.ts	Client-side state management (likely React context or Zustand) for the triage result across the multi-page flow.
triage-session.ts	Session token / local storage management across upload → result → guidance pages.
image-compress.ts	Client-side image compression before upload (using Canvas API).
demo-data.ts	Hard-coded demo scenario data for offline/demo mode.
follow-up-copy.ts	Copy strings for follow-up question prompts.
utils.ts	Small shared utilities (<code>cn()</code> class merging helper).
server/	Server-side utility helpers for Next.js API routes.

Key Frontend Tech Stack

- **Next.js 15** (App Router, RSC)
- **React 19**
- **TypeScript 5.8**
- **MUI v7** + **Radix UI** + **shadcn/ui** primitives
- **TailwindCSS v4**
- **Motion** (Framer Motion v12) for animations
- **Recharts** for any data visualisation
- **Sonner** for toast notifications

/data/ — Dataset and Configuration Files

Path	Contents
<code>data/round2/theme2_rules.json</code>	The live rule file. Version <code>2026-05-theme2-round2-v1</code> . Defines the mapping from each <code>ObservationResultV2</code> key to <code>fault_type_v2</code> , <code>recipient_type</code> , <code>assigned_team_id</code> , <code>action_message</code> , and <code>required_proof_next</code> . Also contains <code>error_log_rules</code> for blinking-red flash codes 6–9.
<code>data/round2/manifest.csv</code>	Dataset manifest of all Round 2 evaluation images with metadata columns.
<code>data/round2/manifest_summary.json</code>	Aggregate statistics over the manifest.
<code>data/round2/evaluation_cases.json</code>	Gold-standard evaluation cases used for <code>blind-image-eval</code> mode.
<code>data/round2/manual_review_cases.json</code>	Quarantined/uncertain cases awaiting human review before promotion to eval.
<code>data/round2/pseudo_label_summary.json</code>	Summary statistics of auto-generated pseudo-labels.
<code>data/demo/scenarios.json</code>	Pre-built demo scenarios (site, charger, hint text, expected outputs) served by <code>/api/v1/demo/scenarios</code> .
<code>data/sites/sites.json</code>	List of known EV charging sites with <code>site_id</code> , <code>site_name</code> , charger config, and capability flags (<code>has_local_resolver</code> , <code>has_remote_ops</code>).

`/scripts/` — Evaluation & Dataset Utilities

Script	Purpose
<code>build_round2_manifest.py</code>	Scans image folders, builds the <code>manifest.csv</code> from filenames and folder-based weak labels (folder names encode component and observation).
<code>check_round2_eval_coverage.py</code>	Verifies that <code>evaluation_cases.json</code> has adequate coverage across all observation result categories.
<code>evaluate_round2_cases.py</code>	Main eval runner. Two modes: <code>weak-label-sanity</code> (deterministic, no Gemini, regression checks) and <code>blind-image-eval</code> (calls the live <code>/triage</code> endpoint with real images, compares to gold labels).
<code>pseudo_label_round2_with_theme2.py</code>	Batch-runs Gemini perception over unlabelled images to generate pseudo-labels for training/analysis.

Script	Purpose
<code>summarize_round2_pseudo_labels.py</code>	Aggregates pseudo-label results into a summary JSON.
<code>promote_manual_review_to_eval.py</code>	Moves human-reviewed cases from <code>manual_review_cases.json</code> into <code>evaluation_cases.json</code> .
<code>extract_round2_video_frames.py</code>	Extracts frames from video evidence files for inclusion in the dataset.
<code>run_unseen_external_smoke.py</code>	Smoke-tests the live backend with images not in the training/eval set.
<code>gemini_smoke_test.py</code>	Minimal Gemini API connectivity test.
<code>run_final_validation.ps1</code>	PowerShell script that chains pytest, pyright, frontend build, and eval checks.
<code>round2_eval_utils.py</code>	Shared utility functions imported by the other eval scripts.

/docs/ — Architecture and Process Documentation

File	Purpose
<code>theme2_runtime_contract.md</code>	The official API contract: full request/response JSON shapes for <code>/triage</code> , supported evidence fields, organizer output definitions, and deprecated Round 1 fields.
<code>deployment_cloudrun_vercel.md</code>	Step-by-step Cloud Run + Vercel deployment guide with <code>gcLOUD</code> commands and environment variable configurations.
<code>round2_evaluation_methodology.md</code>	Explains how weak-label-sanity vs. blind-image-eval modes differ, coverage requirements, and quarantine/promotion policy.
<code>final_demo_runbook.md</code>	Runbook for running the final live demo: startup order, health checks, scenario walk-through, known edge cases.
<code>final_report_metrics_template.md</code>	Template for the final competition/evaluation report with accuracy and recall metrics slots.
<code>round1_archive_handoff.md</code>	Handoff notes explaining what was archived from Round 1 and why.

`/.github/workflows/ci.yml`

GitHub Actions CI pipeline — likely runs pytest and/or the frontend build on push/PR.

/Dockerfile

Builds the FastAPI backend image for Cloud Run deployment from the repo root.

/.dockerignore

Excludes node_modules, .venv, data images, and other large/sensitive files from the Docker context.

Key Design Decisions

Decision	Rationale
Deterministic rule mapper over pure LLM routing	The organizer's "Theme 2 guide" has fixed rules. Using a JSON rule file makes outputs auditable, versionable, and reproducible independent of VLM temperature.
VLM perception + heuristic fallback	Gemini provides rich visual extraction. If unavailable or low-confidence, heuristic logic uses photo_hint and symptom_text to produce conservative results rather than failing.
Override layer on top of rules	Safety keywords (smoke, sparks, burnt), repeated MCB trips, and persistent no-light after breaker check all override the base rule to escalate to after_sales_team. This keeps the happy path simple while handling edge cases.
Audit trail at every pipeline stage	Every call to /triage creates multiple triage_audits rows (intake_preview, theme2_perception, theme2_mapping, triage_result), making debugging and post-hoc analysis straightforward.
Next.js API proxy	All browser-to-backend calls go through Next.js API routes, keeping the FastAPI URL server-side-only and enabling CORS-free operation.
Two eval modes	weak-label-sanity uses only folder-name labels (fast, no Gemini cost, regression guard). blind-image-eval calls the live endpoint with real images (accurate, requires API key).

Infrastructure Summary

Layer	Technology	Hosting
Frontend	Next.js 15 / React 19 / TypeScript	Vercel
Backend	FastAPI + Uvicorn / Python 3.11	Google Cloud Run (rexcharge-backend)
VLM	Gemini (gemini-3-flash-preview)	Google AI API
Database	PostgreSQL 16 (omnitriage DB)	Neon (production) / Docker (local)
File Storage	Base64 → local disk or GCS	GCS bucket rexcharge-uploads-rexcharge-experiment
CI	GitHub Actions	GitHub